



BUILD A WORLD MODEL FROM SCRATCH · LECTURE 3

Your First *World Model*

We build one thing today, completely and from scratch: a neural network that learns how a small game works — so well that at the end, you can play the game *inside the network's imagination*.

EPISODES

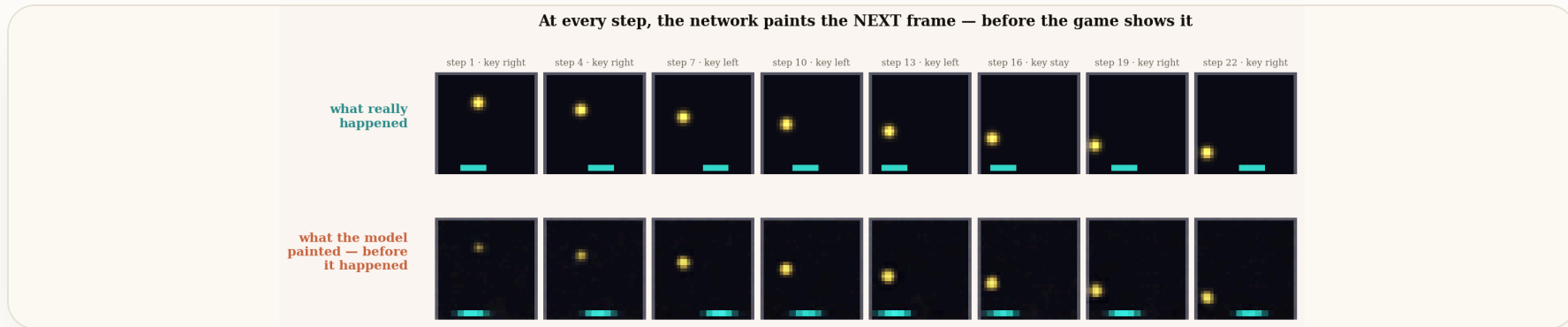
ENCODER

MEMORY

PREDICTION

Vizuara AI Labs · Build a World Model from Scratch

Watch this. The network paints the next frame — before it happens.



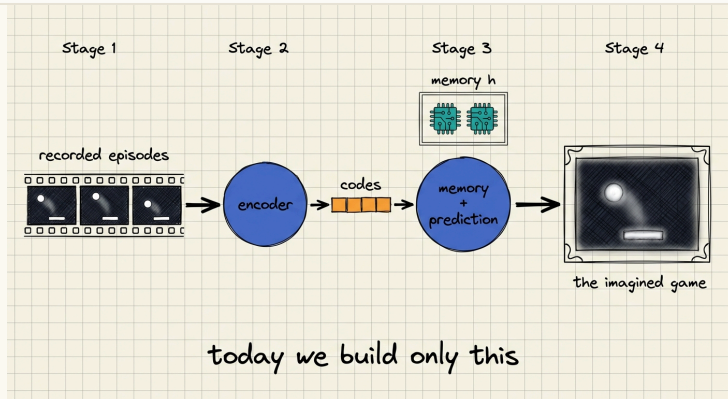
What you are seeing

Top row: the real game. Bottom row: at every step, **a neural network paints what the next frame will look like** — from the previous frames and the player's key press alone, before the game shows it. Ball position, ball direction, paddle response: all correct, at every step. The network learned this world from recorded play.

What today is — and is not

Today we build exactly this and nothing more: **a world model**. No controller, no planner, no rewards — the machinery for *acting well* comes later in the series. Today's single question: *how does a network learn a world?*

The whole lecture on one picture



Part 1 • Training

Collect episodes → compress frames into codes (network V) → learn to predict the next code with a memory (network M). Step by step, one figure per step, with the exact tensors at every stage.

Part 2 • Deployment

Switch the game off. Run the trained model in a closed loop, feed it YOUR actions — and play inside it. Including the honest engineering it took to make that stable.

Explanation from Rajat

— WHITEBOARD —



PART 1 OF 2

Training

Where the data comes from, what the networks are, which frame is used at which timestep, and where exactly the loss is computed.

EPISODES · ENCODER · MEMORY · LOSS

The world: MiniPong

Our world for today is a small Pong-like game we wrote in ~60 lines:

● The screen

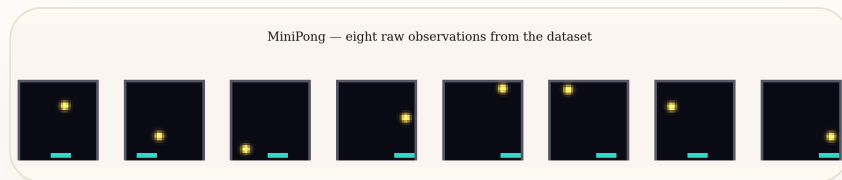
32×32 pixels, 3 colour channels — every frame is $32 \times 32 \times 3 = 3,072$ numbers.

● The objects

A paddle at the bottom that YOU move (3 actions: left · stay · right), and a ball that flies one pixel per step and bounces off the walls — and off the paddle, which can bend its direction.

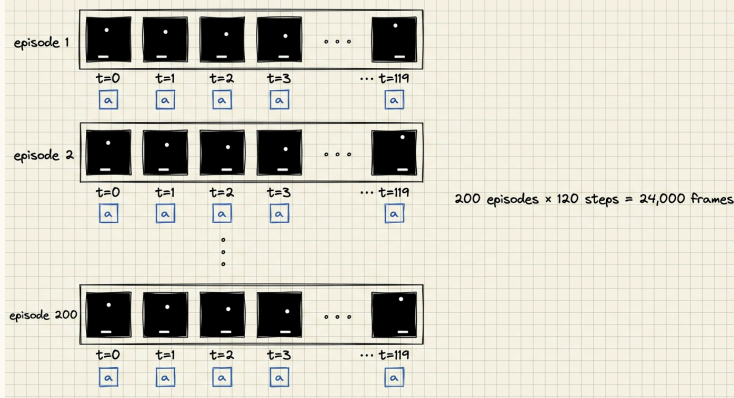
● The catch — remember this

Look at any single frame: you can see WHERE the ball is. You cannot see *where it is going*. That missing piece of information will shape the entire architecture.



Eight raw observations. Bright ball, bright paddle, dark world — 3,072 numbers each.

The dataset: 200 recorded episodes



What we record

We let a **random player** play 200 episodes of 120 timesteps. At every timestep we store exactly two things: the **frame** (what the screen showed) and the **action** taken. Total: $200 \times 120 = 24,000$ frame-action pairs. Nothing else — no labels, no rewards, no positions.

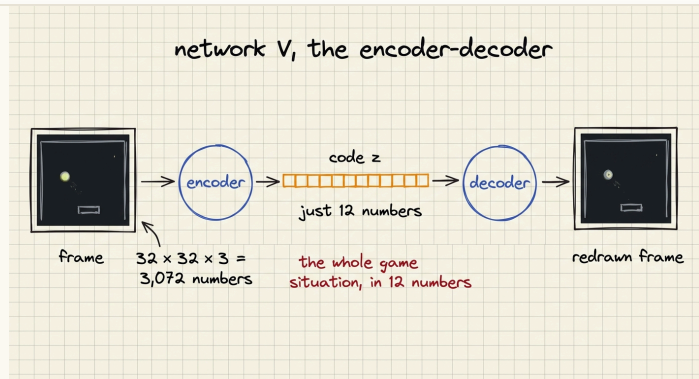
Why a RANDOM player?

The world model's job is to learn *how the world works* — and physics doesn't care whether the player is skilled. One subtlety: our random player **holds each key for a while** before switching, like a human would. At deployment a player holds keys too — training data must visit the situations deployment will.

Explanation from Rajat

— WHITEBOARD —

Network V: compress every frame into a code



Why compress at all?

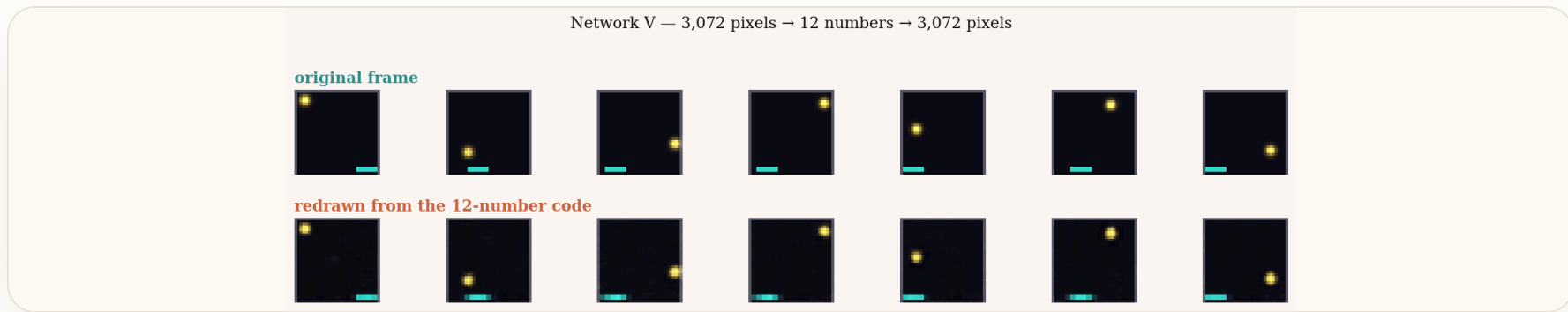
A frame is 3,072 numbers, but the *situation* is tiny: ball position, paddle position. The **encoder** squeezes each frame into a **code z of just 12 numbers**; the **decoder** proves nothing was lost by redrawing the frame from the code alone. Trained on all 24,000 frames with one signal: *make the redrawn frame match the original*.

Two details that matter later

During training we add a little noise around each code before decoding — soon another network will *predict* codes, and its predictions land *near* real codes, never exactly on them; near a code must still decode to a sane frame. And consecutive frames are pushed to get *nearby* codes: the world moves smoothly, so the codes must too.

We trained V — and hit our first honest lesson

The experiment: train network V on the 24,000 frames (332 seconds on a laptop CPU), then ask it to redraw held-out frames from their 12-number codes:

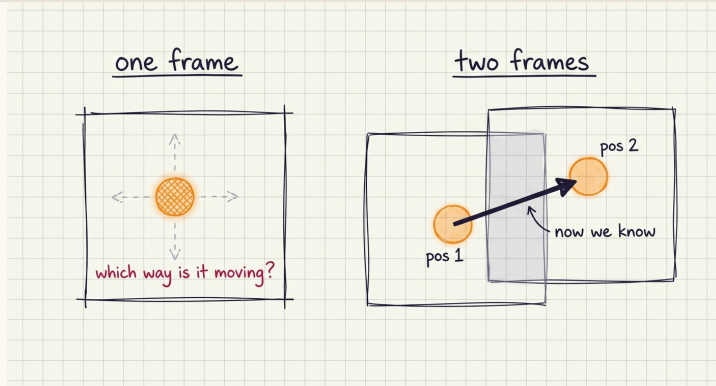


The lesson we hit on the first try: our first V redraw the paddle perfectly and **completely dropped the ball**. Why? The ball is 4 bright pixels out of 1,024 — to a plain reconstruction loss, ignoring it costs almost nothing. The fix is one line: *weight bright pixels more heavily in the loss*. Compression keeps what the loss pays for — nothing else. You will meet this lesson in every world model you ever build.

Explanation from Rajat

— WHITEBOARD —

Before the next network: why one frame is not enough



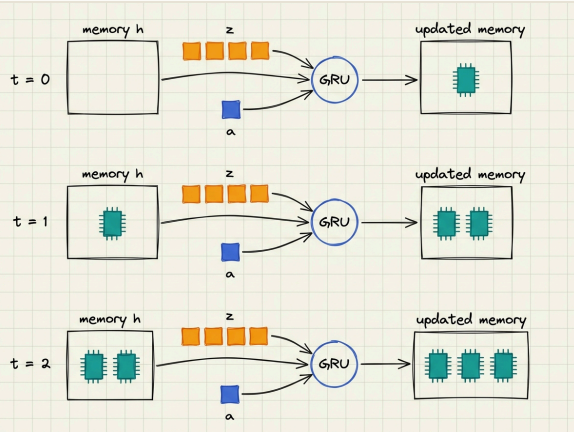
The problem, precisely

We want to predict the NEXT frame. But the current frame alone cannot tell us: the ball at position (12, 9) might be moving down-left or down-right – both look identical *right now*. The information we need – velocity – lives in the *difference between consecutive frames*, not in any single one.

The consequence for our architecture

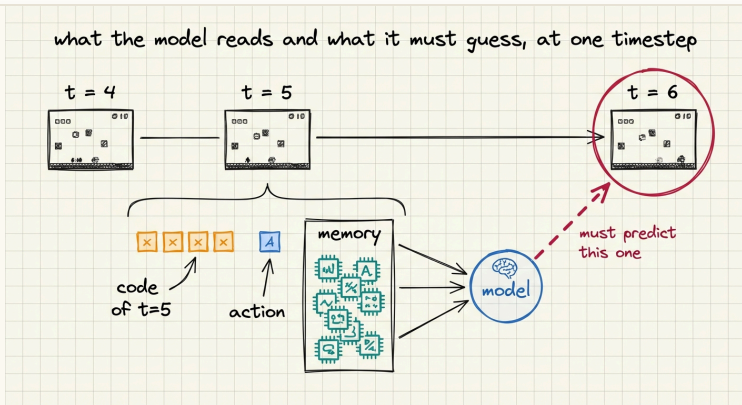
The prediction network cannot be a simple frame-in, frame-out function. It needs a **memory**: a vector that persists across timesteps and accumulates what single frames cannot show. This is the single most important design decision of the lecture.

The memory: updated at every timestep



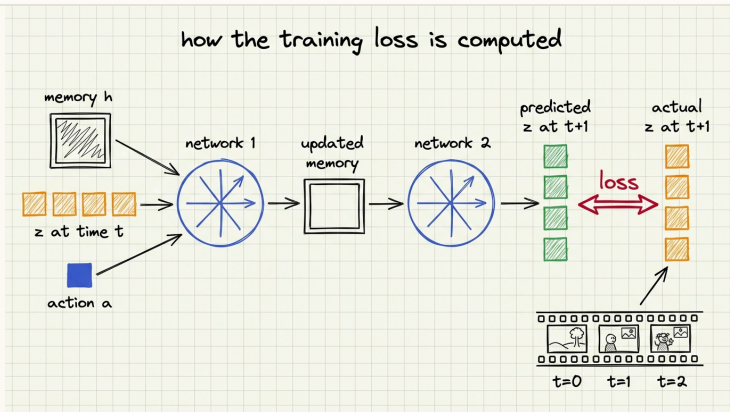
Read it row by row. At every timestep, one network (a GRU — 128 memory numbers in our build) takes three inputs: **the memory as it was**, **the current code z** , and **the action a** — and produces the updated memory. At $t=0$ the memory is empty. After one step it holds one fact (where things are). After two steps it holds the fact no frame ever showed: **which way the ball is moving** — because it has seen two positions and can keep their difference. The memory is never wiped during an episode; only rewritten.

At one timestep: what the model reads, and what it must guess



Fix this picture in your mind — it answers the bookkeeping questions precisely. At timestep $t = 5$ the model reads three things: the **code of frame 5** (from network V), the **action taken at step 5**, and its own **memory** (which has digested steps 0–4). It must then output a guess for the **code of frame 6** — a frame it has not seen. Frame 6's real code, computed by V, is the answer key. This repeats at every timestep of every episode: 24,000 small exams.

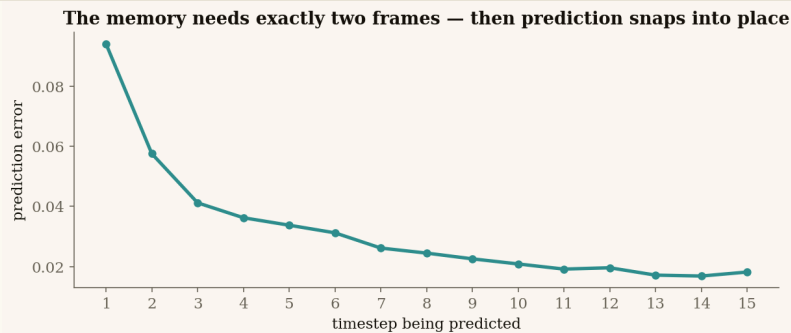
So when is the training happening – and what is trained?



Here is the full training step. **Network 1** (the GRU) blends memory + code + action into the updated memory. **Network 2** (a small two-layer head) reads the updated memory and outputs the **predicted next code** (green). The real next frame goes through V to give the **actual next code** (orange). The **loss** is the distance between green and orange — and its gradient trains *both* networks together, at every timestep, across all 200 episodes, epoch after epoch. That's it. That is how a network learns a world.

The result that proves the memory story

The experiment: after training (46 seconds), measure the prediction error separately at each timestep of fresh episodes. If the memory story is true, the error should be LARGE at $t = 1$ — velocity is unknowable from one frame — and small from $t = 2$ onward, once the memory has seen two positions:



Exactly as predicted: error 0.094 at the first step — then it collapses to 0.041 by step 3 and stays there. The network was never told about velocity. It discovered that keeping the difference between two positions is the most useful thing a memory can do — because prediction is what the loss pays for.

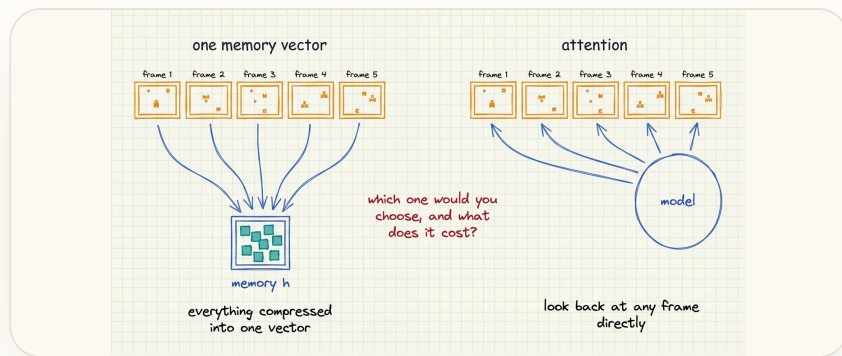
Two questions you should be asking right now

1 "Why two networks? Can't we do it in one?"

You can — nothing stops you from training one network that maps (frame, action, memory) → next frame directly. We split it because each piece is reusable and inspectable: V's codes can be studied on their own, M can be swapped without retraining vision. And a preview: the next lecture merges them back into *one jointly-trained model* — and shows exactly what that buys.

2 "Why squeeze history into ONE vector? Why not attention?"

Excellent instinct — that is precisely what a Transformer would do: keep *all* past frames and let the model look back at any of them directly. The trade: attention's memory and compute grow with history length; the recurrent vector stays constant-size forever. Modern world models use both, and we will meet the attention-based ones later in the series.



Two ways to remember: compress everything into one vector, or keep everything and look back.

Explanation from Rajat

— WHITEBOARD —



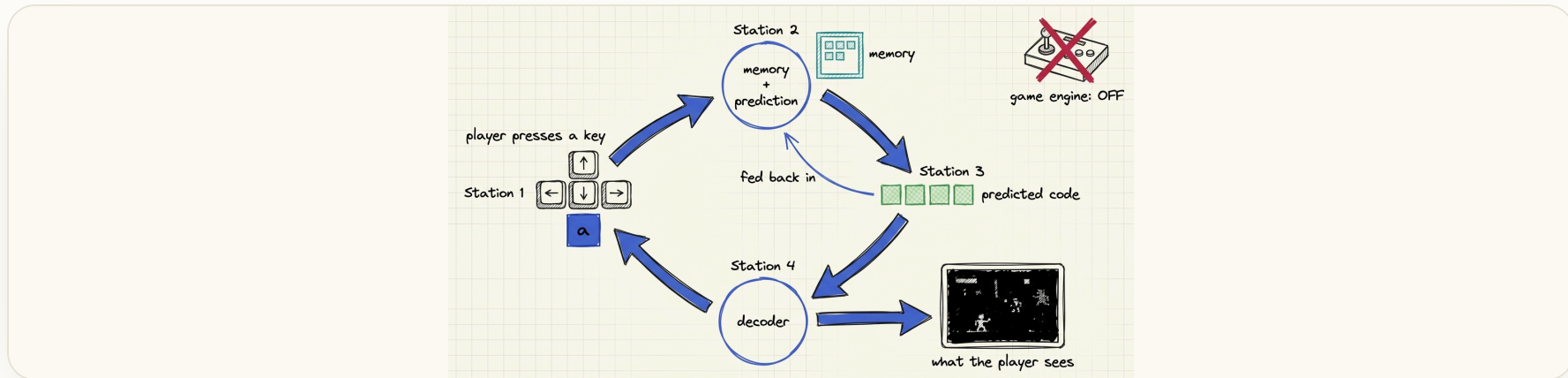
PART 2 OF 2

Deployment

Switch the game off. The model now has to survive on its own predictions — with your hands on the keys.

THE CLOSED LOOP

Deployment: the model eats its own predictions



During training, the model received a *real* frame's code at every step. Now there are no frames — the game engine is off. So the loop changes in one crucial way: **the predicted code is fed back in as if it were real**. The player presses a key → networks 1 and 2 produce the predicted next code → the decoder paints it → the player sees the painted frame → the prediction becomes the next step's input. Around and around. The model is now the game.

Surviving the closed loop took real engineering

Training in open loop and running in closed loop are different sports. Our first deployed model kept the paddle perfect — and lost the ball within a few frames. The fixes, in the order we found them:

1 Every code dimension must matter equally

The paddle dominates the code's variance, so the loss barely noticed errors in the ball's dimensions. Fix: normalise each of the 12 code dimensions to the same scale before training network M — and weight the ball's pixels up in V's loss, or compression happily drops the smallest, most important thing on screen.

2 Practise the closed loop during training

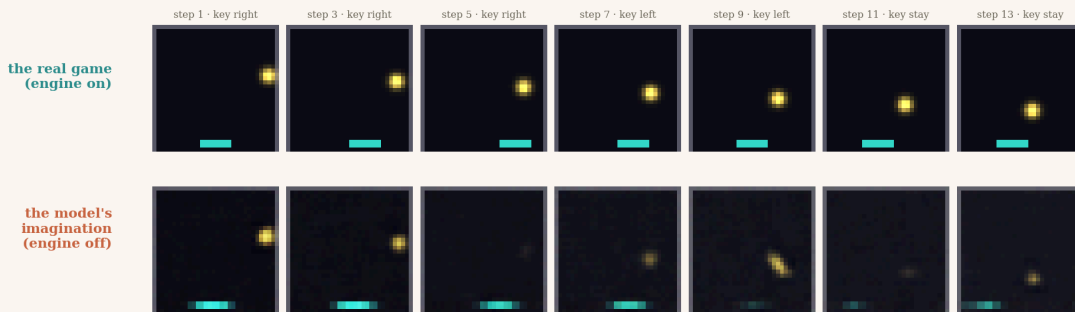
After normal training, fine-tune M on *closed-loop* rollouts — it must predict several steps ahead while eating its own outputs, and each predicted code is decoded back to pixels and compared against the real frame. Predict the *change* in the code, not the code itself: when the model is unsure, "nothing moves" is a far safer guess than "fade everything toward the average".

3 The world itself must be learnable

Two of our "obviously fine" world-design choices were quietly fatal. Random ball respawns: a deterministic predictor facing true randomness predicts the average of all outcomes — an invisible smear. And integer-pixel motion: positions that snap to whole pixels give the codes isolated islands, and a prediction landing between islands paints nothing. Deterministic physics + smooth sub-pixel motion fixed both.

The closed loop, for real — and what it revealed

The same keys, two worlds — one of them is a neural network's dream



Read the strip

Top row: the real engine. Bottom row: the model's imagination, fed the same key presses and *its own predictions* — no frames from the game. The paddle obeys every key for as long as you play. The ball is painted for the first steps, then fades and re-forms as the dream drifts. Total machinery: two networks, 167,527 parameters, trained in under ten minutes on a laptop.

The honest limit — and the cliffhanger

One-step prediction is near-perfect (you saw it in the opening demo). But in a closed loop every tiny error is eaten and amplified at the next step, and within a handful of steps the ball dissolves. This compounding-error problem is THE central problem of world models — and solving it is exactly where this series goes next.

Explanation from Rajat

— WHITEBOARD —

Build it yourself — the notebook is the lecture

One notebook, the full build

`pong_worldmodel.ipynb` — MiniPong itself (~60 lines), the dataset collection, network V, network M with the memory, the per-timestep error experiment, and the playable closed loop.

Free Colab, under ten minutes end to end

Every number and every plot on today's slides is this notebook's real output. Change a line, rerun, watch the world change.

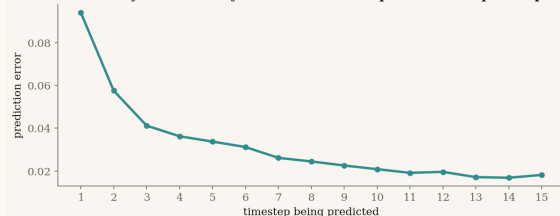
Three exercises

① Shrink the code to 3 numbers — what breaks first? ② Zero the memory at every step and watch prediction fail exactly where the theory says. ③ Re-enable random ball respawns and watch the deterministic predictor smear — then explain why.

Link in the description

Open in Colab, press "Run all", then play inside your own model.

The memory needs exactly two frames — then prediction snaps into place



Network V — 3,072 pixels → 12 numbers → 3,072 pixels



Your outputs will look like these — same code, your machine.

Four things to keep

1 A world model = compress + remember + predict

Encoder to a small code, a memory updated every timestep, a head that predicts the next code. Three jobs, two small networks, one loss.

2 Compression keeps what the loss pays for

The ball vanished until we made the loss care about it. Watch for this in every model you ever train.

3 Memory earns its keep in exactly two frames

Prediction error 0.094 with one frame of history, 0.041 with three — velocity lives in the difference between frames, and the memory learned to keep it unprompted.

4 Deployment is a different sport from training

A model that predicts one step well can still collapse when it eats its own predictions. Practise the closed loop; respect true randomness.

Next: making the model useful

You now own a network that contains a playable copy of its world — and you watched that copy drift when pushed too far. The next question writes itself: **how do we make dreams that last?**

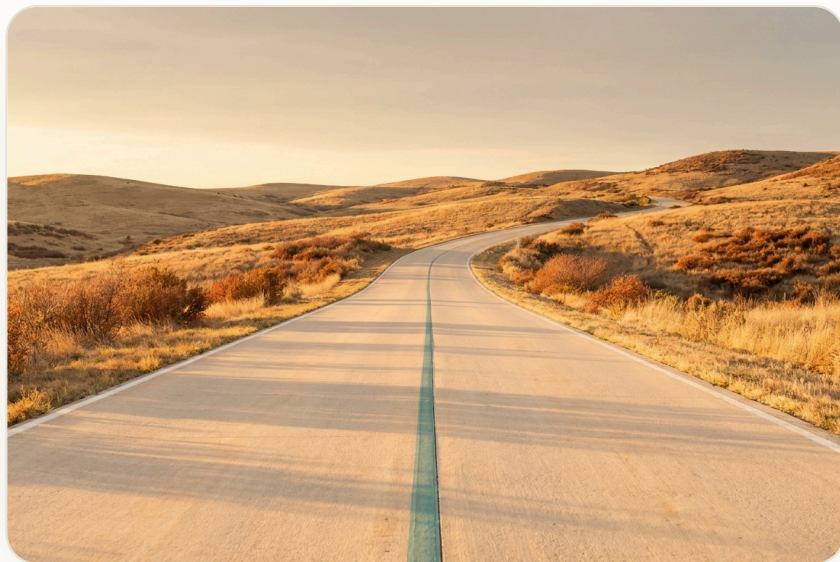
● Better models

Our encoder never knew its codes would be used for prediction — we trained V and M separately, then paid for it. Next lecture's model trains them *together*, built from the ground up for long, stable dreams.

● Then: acting

Using imagined futures to choose actions, plan manoeuvres, and eventually train whole agents inside the model — the planning part of our taxonomy, later in the series.

●●● Vizura AI · Build a World Model from Scratch



First we learned the world. Next we put it to work.